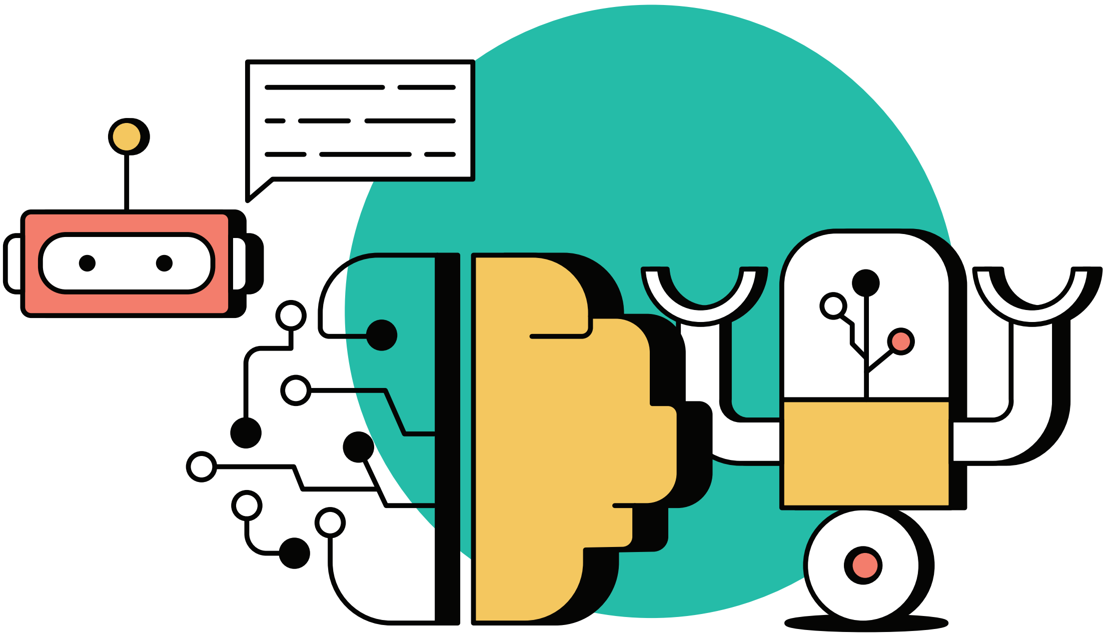
جزوه رشته هوش مصنوعی



استاد	محمد بهجت پور
درس	یادگیری ماشین
فصل	دسته‌بندی
مبحث	الگوریتم پرسپترون

به نام خدا

مسئله دسته‌بندی

D مجموعه داده → $D=\{(x_i, y_i)\}_{i=1}^{N}$

x بردار ویژگی → $x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$ $x \in R^n$ → بردار x، n بعدی است.

بردار x، n مؤلفه دارد

y_i: برچسب داده iام → $y_i \in \{\underset{-1}{0}, \underset{+1}{1}\}$ → دسته‌بندی دوکلاسه یا Binary Classification

→ One vs Rest

One v's One

قطر اصلی ماتریس A

$A = \begin{bmatrix} ۱ & -۱ \\ ۳ & ۲ \end{bmatrix}_{۲\times۲} \rightarrow A^T = \begin{bmatrix} ۱ & ۳ \\ -۱ & ۲ \end{bmatrix}_{۲\times۲}$

$a_{ij} \rightarrow a_{ji}$

Transpose ← ترانهاده‌کردن ماتریس/بردار → قرینه کردن ماتریس نسبت به قطر اصلی. مثال:

سطری ← ستونی / ستونی ← سطری. مثال:

$v = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} \rightarrow v^T = [v_1 \quad v_2 \quad v_3]$

ایده اصلی پرسپترون: ابتدا ویژگی‌ها را در وزن های مدل ضرب می‌کنیم و حاصل را با بایاس (Bias) جمع می‌کنیم:

$$z_i = w^T \cdot x_i + b$$

نماد ترانهاده

بایاس

بردار وزن ها

نماد ضرب داخلی دو بردار

بردار ویژگی نمونه iام

$y = ax + b$ شیب خط، عرض ازمبدا

$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} \rightarrow w^T = [w_1 \quad w_2 \; .... \; w_n]$

وسپس اگر حاصل مثبت بود، مدل یک کلاس را پیش‌بینی می‌کند و اگر صفر یا منفی بود، کلاس دیگر را:

$$h(z) = \begin{cases} z_i \leq ۰ \rightarrow \text{کلاس } -۱ \\ ۰ < z_i \rightarrow \text{کلاس } +۱ \end{cases}$$

$w^T x + b = ۰$ → یک خط است ⇐ مرز تصمیم مدل خطی است

$$y_i = h(w^T x_i + b)$$

کلاس پیش‌بینی شده برای نمونه iام.

تفسیر معانی وزن ها: مثلاً در $z = \overset{w_1}{۳}x_1 - \overset{w_2}{۲}x_2 + \underset{b}{۴}$

پس وزن مثبت یعنی افزایش ویژگی متناظر با آن، باید z را افزایش دهد.

و وزن منفی یعنی افزایش مقدار ویژگی متناظر با آن، باید مقدار z را کاهش دهد.

و بزرگی وزن نشان می‌دهد آن ویژگی چه مقدار روی مقدار خطی z اثر می‌گذارد. (البته این تفسیر به مقیاس ویژگی‌ها نیز وابسته است)

$y = ax + b \rightarrow d: y = ax \rightarrow (۰,۰) \in d$ → خط d مبدأ گذراست.

کاربرد Bias در مدل: اگر Bias نداشتیم، مدل به صورت زیر می‌بود:

$z = w^T \cdot x \rightarrow z = w_1 x_1 + w_2 x_2$ (در حالت دو بعدی) → این خط لزوماً از مبدأ مختصات عبور می‌کند. اما با افزودن b به z، خط می‌تواند جابجا شود و لازم نیست از مبدأ عبور کند.

در نتیجه Bias آزادی بیشتری برای تعیین مرز تصمیم به مدل می‌دهد.

مرز تصمیم: به طور کلی مرز تصمیم جایی است که مدل بین دو کلاس مردد است. در پرسپترون این مرز زمانی اتفاق می‌افته که $z = 0$ شود. یعنی $w^T x + b = 0$.

مثلاً اگر داشته باشیم $2x_1 + 3x_2 - 6 = 0$، مرز تصمیم مدل همین خط است، و اگر $2x_1 + 3x_2 - 6 < 0$ باشد نمونه به کلاس $-$ و اگر $2x_1 + 3x_2 - 6 > 0$ باشد، نمونه به کلاس $+$ تعلق دارد.

پس پرسپترون، فضای ویژگی (Feature Space) را به ۲ بخش تقسیم می‌کنه.

در دو بعد مرز تصمیم خط است. در سه بعد صفحه است و در ابعاد بالاتر Hyper Plain یا ابرصفحه است.

مثال: فرض کنید پرسپترون روبرو را داریم، نمونه $x = (8, 3, -2)$ به کدام دسته تعلق دارد؟

$w = \begin{bmatrix}2\\-1\\4\end{bmatrix} \quad b = -3$

$z = w^T . x + b \longrightarrow [2\ \ -1\ \ 4] . [8\ \ 3\ \ -2] - 3 = \underset{8}{\cancel{16}}\ \cancel{-3}\ \underset{-6}{\cancel{-8}} - 3 = 2$

$y = h(z)$

$\hookrightarrow h(z) = h(2) = +1 \rightarrow$ پس این نمونه به کلاس $+1$ تعلق دارد.

فرآیند یادگیری مدل: در یادگیری ماشین ما وزن‌ها را نمی‌دانیم، ولی در مرحله نخست مقادیر آنها را به صورت تصادفی تعیین می‌کنیم.

مثلاً: $w = \begin{bmatrix}-3.5\\4.9\end{bmatrix}, b = 9.1$ یا $w = \begin{bmatrix}1.4\\3.5\end{bmatrix}, b = -0.2$

سپس داده‌های آموزش را یکی یکی به مدل می‌دهیم. اگر مدل یک نمونه را درست دسته‌بندی کرد کاری نمی‌کنیم، اما اگر مدل یک نمونه را نادرست طبقه‌بندی کرده بود وزن‌ها را تغییر می‌دهیم.

$w \leftarrow w + \overset{\eta}{\alpha}\, y_i x_i \qquad b \leftarrow b + \alpha y_i \qquad y_i \in \{-1, +1\}$

$\alpha \rightarrow$ نرخ یادگیری

↳ هایپرپارامتر ← برای شروع می‌توانیم $\alpha = 0.1$ یا $\alpha = 0.01$ یا ساده‌تر $\alpha = 1$ بگذاریم.

مثال: فرض کنید داده‌هایی مطابق جدول زیر به ما داده شده است. وزن‌های مدل را از $b = 0$ و $w = (0, 0)$ یک مرحله به‌روزرسانی کنید؟

x_1	x_2	y
2	1	+1
1	2	−1

$\longrightarrow w \leftarrow \begin{bmatrix}0\\0\end{bmatrix} + 0.1 \times 1 \times \begin{bmatrix}2\\1\end{bmatrix} \rightarrow w = \begin{bmatrix}0.2\\0.1\end{bmatrix}$ Epoch 1

$b \leftarrow 0 + 0.1 \times +1 \rightarrow b = 0.1$

$w \leftarrow \begin{bmatrix}0.2\\0.1\end{bmatrix} + 0.1 \times (-1) \times \begin{bmatrix}1\\2\end{bmatrix} \rightarrow \begin{bmatrix}0.1\\-0.1\end{bmatrix} \longrightarrow$ وزن‌های نهایی

Epoch 2

$b \leftarrow 0.1 + 0.1 \times (-1) \rightarrow b = 0 \rightarrow$ بایاس نهایی

صورت الگوریتم پرسپترون:

ورودی: ۱- داده های آموزشی: x_i ۲- برچسب ها: y_i
۳- نرخ یادگیری (Learning Rate): α ۴- تعداد epoch ها

مراحل: ۱- مقداردهی اولیه وزن ها ۲- مقدار دهی اولیه Bias

۳- برای هر نمونه داده آموزشی:
۳٫۱- محاسبه $z = w^T x_i + b$
۳٫۲- به روزرسانی وزن ها اگر $y_i \times h(z) \leqslant 0$

۴- بررسی همه نمونه‌ها و در صورت لزوم تکرار مرحله ۳ برای چند epoch.
+ اگر در یک epoch همه نمونه‌ها به درستی دسته‌بندی شده بودند، فرآیند آموزشی را متوقف می‌کنیم.

epoch چیست؟ ← به هربار بررسی تمامی داده‌های آموزشی، یک epoch گفته می‌شود. مثلاً اگر ۱۰۰ داده آموزشی داشته باشیم، epoch ها به صورت روبرو خواهند بود:

Epoch 1: هر ۱۰۰ نمونه یک بار بررسی می‌شوند.
Epoch 2: هر ۱۰۰ نمونه دوباره بررسی می‌شوند.
Epoch 3: . . . .
. . . .
. . . .

هدف این است تا در نهایت به نقطه‌ای برسیم که همه نمونه‌های آموزشی به درستی طبقه‌بندی شوند.

آیا پرسپترون همیشه همگرا می‌شود؟ خیر - اگر داده‌ها به صورت خطی جداپذیر باشند این الگوریتم پس از تعداد محدودی آپدیت به یک جداکننده خطی می‌رسه. اما اگر داده‌ها به صورت خطی جداپذیر نباشند، پرسپترون دائماً وزن‌ها را تغییر می‌دهد اما نمی‌تواند همه داده‌ها را درست طبقه‌بندی کند.

این موضوع به Perceptron Convergence Theorem معروف است.

معنی جداپذیری خطی: اگر بتوانیم خط/صفحه/ابرصفحه‌ای بیابیم که دو کلاس مثبت و منفی را از هم جدا کند، داده‌ها جداپذیر خطی هستند.

مثال XOR: داده‌های جدول روبرو را نمی‌توان به صورت خطی جدا کرد. بنابراین این داده‌ها را نمی‌توان با یک پرسپترون ساده دسته‌بندی کرد.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

در پرسپترون کلاسیک از تابع فعالسازی پله‌ای استفاده می‌شود اما در شبکه عصبی مدرن از توابعی مثل Sigmoid، Softmax، Tanh، ReLU و ... استفاده می‌شود.